

Documentação do Projeto Gincana (React + Express + Firebase)

Este documento consolida toda a estrutura do projeto, cobrindo desde o setup até o deploy, seguindo boas práticas de desenvolvimento com arquitetura em camadas. O sistema é composto por um frontend em React (Vite) e um backend em Express.js, com banco de dados Firebase (Firestore).

Setup do Projeto

O projeto utiliza **Node.js** e **Express** no backend, **React com Vite** no frontend, e **Firebase (Firestore)** como banco de dados principal.

Dependências do Backend

Pacote	Versão	Função no projeto
bcrypt	5.1.1	Hash de senhas e comparação segura
dotenv	16.6.1	Carrega variáveis de ambiente do arquivo .env
express	4.21.2	Servidor HTTP e roteamento de requisições
firebase-admin	11.11.1	Integração com Firebase / Firestore
jsonwebtoken	9.0.2	Geração e validação de tokens JWT
nodemon	2.0.22	Reinício automático do servidor durante desenvolvimento

Pré-requisitos

- Node.js LTS (≥ 18)
- npm ou yarn
- Git

Instalação

1. Clonar o repositório:

```
git clone <URL>
cd gincana
```

2. Instalar dependências do backend:

```
cd backend  
npm install
```

3. Instalar dependências do frontend:

```
cd ../frontend  
npm install
```

Execução em modo desenvolvimento

Backend (Express.js)

```
cd backend  
npm run dev
```

Frontend (React + Vite)

```
cd frontend  
npm run dev
```



Arquitetura do Projeto

A arquitetura segue o padrão de camadas, separando responsabilidades entre controle, serviço e repositório. O backend foi desenvolvido em **Express.js**, escolhido pela leveza e simplicidade, com Firebase (Firestore) como banco de dados.



Frontend

- React + Vite
- Context API para gerenciamento de estado global
- TailwindCSS para estilização
- React Router para roteamento



Backend

- Express.js (alternativas consideradas: NestJS e Fastify)
- Estrutura modular: controllers, services e repositories
- Autenticação baseada em JWT
- Integração com Firebase (Firestore)



Banco de Dados

- Banco principal: Firebase (Firestore)
- Estrutura de dados ainda em definição



Estrutura do Projeto (Frontend + Backend)

Estrutura	Descrição
backend/	Pasta raiz do backend
├─ src/	Código fonte principal do backend
└─ config/	Configurações do app, banco, JWT, .env
└─ routes/	Definição de endpoints
└─ controllers/	Entrada das requisições

└─ services/	Regras de negócio
└─ repositories/	Acesso ao banco de dados
└─ models/	Entidades/ORM
└─ middlewares/	Auth, validações, tratamento de erros
└─ utils/	Helpers, formatação, funções auxiliares
└─ app.js	Inicialização do servidor
└─ tests/	Testes unitários e de integração
└─ package.json	Configurações do projeto e dependências
└─ .env	Variáveis de ambiente
Estrutura	Descrição
frontend/	Pasta raiz do frontend (React + Vite)
└─ src/	Código fonte principal do frontend
└─ components/	Componentes reutilizáveis
└─ pages/	Páginas e rotas
└─ hooks/	Custom hooks
└─ services/	Chamadas à API / integração com backend
└─ context/	Context API para estado global
└─ assets/	Imagens, ícones, estilos

◆ Boas práticas no frontend

- Componentes pequenos e reutilizáveis
- Pastas por funcionalidade quando necessário
- Padronização com ESLint + Prettier
- Versionamento semântico de commits



Style Guide



Lint e formatação

- ESLint para padronização de código JS/TS
- Prettier para formatação consistente
- EditorConfig para unificar indentação
- Husky + lint-staged para pre-commit



Organização de pastas

gincana/
├── backend/ → API e lógica do servidor
│ └── src/
└── frontend/ → Interface React
 └── src/



Convenções de escrita

- Identação: 2 espaços
- Aspas simples
- Ponto e vírgula omitido
- Imports absolutos
- Nomes de arquivos: kebab-case.js no frontend, PascalCase.ts para componentes React



Nomes de branches

- feature/: novas features
- fix/: correções leves
- hotfix/: correções urgentes



Clonando o Repositório

```
cd existing_repo
git remote add origin https://gitlab.unipampa.edu.br/ales/rp-vi-2025-2/grupo-01.git
git branch -M main
git push -uf origin main
```

Fluxo de Branches

Branch	Finalidade	Origem	Merge para
main	Código de produção	—	—
develop	Integração contínua	main	release
feature/*	Novas funcionalidades	develop	develop
fix/*	Correções não críticas	develop	develop
hotfix/*	Correções urgentes em produção	main	main → develop

 Commits diretos em main e develop são proibidos. Sempre usar Pull Requests.

Commits Semânticos

Formato:
<type>(<scope>): <mensagem curta no imperativo>

Tipos aceitos:

- feat – nova funcionalidade
- fix – correção de bug
- docs – alterações de documentação
- style – ajustes de formatação
- refactor – refatoração
- test – testes
- build – mudanças de dependências



Contribuição



Como contribuir

1. Crie uma branch a partir de develop:
git checkout develop
git pull origin develop
git checkout -b feature/nome-da-feature
2. Desenvolva seguindo o Style Guide.
3. Faça commits semânticos conforme o Workflow.
4. Envie sua branch:
git push origin feature/nome-da-feature
5. Abra um Pull Request.



Checklist do PR

- Código formatado (ESLint/Prettier)
- Testes rodando localmente
- Documentação atualizada
- Nenhum erro crítico no lint



Deploy

O deploy do projeto é realizado de forma separada entre frontend e backend, garantindo flexibilidade, desempenho e melhor escalabilidade geral do sistema.



Estrutura de Deploy

O frontend é hospedado na **Vercel**, onde o aplicativo React + Vite é compilado e distribuído automaticamente a partir do repositório Git.

O backend é hospedado no **Render** (ou alternativamente no Railway), onde o servidor **Express** é executado de forma completa, responsável pelas rotas, autenticação e regras de negócio.

O banco de dados é o **Firebase (Firestore)**, utilizado tanto para armazenamento de dados quanto para autenticação e integrações com o Firebase Admin SDK.



Processo

1. Frontend (Vercel):

O repositório é conectado ao painel da Vercel, que realiza o build automaticamente a cada novo commit na branch principal.

O diretório raiz do frontend é definido como `frontend`, o comando de build utilizado é `npm run build`, e a pasta de saída é `dist`.

Todas as variáveis de ambiente do frontend, como URLs da API e credenciais do Firebase, são configuradas diretamente no painel da Vercel.

2. Backend (Render):

No painel do Render, cria-se um novo serviço web apontando para o mesmo repositório.

O diretório raiz do backend é definido como `backend`, e o comando de inicialização é `npm run start`.

As variáveis de ambiente necessárias, como `JWT_SECRET`, chaves do Firebase e URLs externas, também são configuradas no painel do Render.

Após o deploy, o Render disponibiliza uma URL pública da API (por exemplo: <https://backend-projeto.onrender.com>).

3. Integração:

No frontend, é configurada a variável de ambiente `VITE_API_URL` apontando para o backend hospedado no Render.

Dessa forma, a comunicação entre o site e a API ocorre de forma segura via HTTPS, garantindo isolamento entre camadas e facilidade de manutenção.



Ambientes

- **Dev:** ambiente local utilizado durante o desenvolvimento, executado com `npm run dev` tanto no frontend quanto no backend.
- **Homolog:** ambiente temporário para testes internos, refletindo as condições reais de produção.
- **Prod:** ambiente de produção final, com o frontend publicado na Vercel e o backend ativo no Render.



Monitoramento e Logs

O frontend hospedado na Vercel possui logs e histórico de builds acessíveis diretamente pelo painel da plataforma.

O backend no Render oferece logs em tempo real, histórico de execução e monitoramento de uptime automático.

O Firebase fornece o console de gerenciamento de dados, autenticações, regras de segurança e monitoramento de atividades do Firestore.



Boas práticas

Para garantir segurança e estabilidade, todas as configurações sensíveis devem ser armazenadas em variáveis de ambiente, nunca diretamente no código-fonte.

Tanto a Vercel quanto o Render já oferecem HTTPS por padrão, o que assegura uma comunicação criptografada entre os serviços.

Recomenda-se manter a branch `main` como base para os deploys automáticos, assegurando consistência entre os ambientes de desenvolvimento e produção.